

SEMESTER END EXAMINATION – 2025-26
Subject: Design and Analysis of Algorithms (DAA)

Subject Code: BCE26PE02 | Class: T.Y. B.E. (Computer Engineering) | Semester: VI

Instructions to Students:

1. All questions are compulsory.
2. Figures to the right indicate full marks.
3. Assume suitable data wherever necessary.
4. Draw neat diagrams and show all steps for numerical problems.

Sample Paper 1

Q1 [Divide and Conquer | Time Complexity]

[7 Marks]

(a) Apply the Merge Sort algorithm to sort the following array step-by-step. Show all recursive splits and merge operations. Also, derive the recurrence relation and compute its time complexity using the Master Theorem. Array: [38, 27, 43, 3, 9, 82, 10]

[4 Marks]

(b) Differentiate between best-case, worst-case, and average-case time complexities of Quick Sort. Under what conditions does Quick Sort degrade to $O(n^2)$?

[3 Marks]

Q2 [Greedy Algorithms | Graph Algorithms]

[8 Marks]

(a) Apply Dijkstra's Algorithm on the following graph and find the shortest path from source vertex A to all other vertices. Show the step-by-step table. Edges:

A→B(2), A→C(4), B→C(1), B→D(7), C→D(3), C→E(5), D→E(2)

[4 Marks]

(b) Solve the Fractional Knapsack problem using the Greedy method for the following instance: Capacity: $M = 15$ Items — Weights: (2, 3, 5, 7, 1, 4, 1) Profits: (10, 5, 15, 7, 6, 18, 3) Show the profit/weight ratios, selection order, and maximum profit obtained.

[4 Marks]

Q3 [Dynamic Programming | Graph Algorithms]

[7 Marks]

(a) Solve the following 0/1 Knapsack problem using Dynamic Programming. Draw the complete DP table and identify the selected items. Knapsack Capacity: $W = 5$

Items — Weights: (2, 3, 4, 2) Values: (3, 4, 5, 3)

[4 Marks]

(b) Explain the Floyd-Warshall algorithm with its recurrence relation. State the time complexity and mention one advantage it has over Dijkstra's algorithm.

[3 Marks]

Q4 [Backtracking | Branch and Bound | Graph Algorithms]
[8 Marks]

(a) Solve the following 0/1 Knapsack problem using the Backtracking approach. Draw the state-space tree, prune infeasible nodes, and identify the optimal solution.
Knapsack Capacity: $M = 7$ Items — Weights: (5, 3, 4, 2) Profits: (20, 15, 12, 10)

[4 Marks]

(b) Apply the graph coloring algorithm using Backtracking for the graph with 4 vertices and the following edges: (1-2), (1-3), (2-4), (3-4). Use 3 colors. Show the assignment step-by-step and state the final valid coloring.

[4 Marks]

Sample Paper 2

Q1 [Time Complexity | Asymptotic Analysis] [7 Marks]

(a) Analyze and determine the upper bound (Big-O), lower bound (Omega), and tight bound (Theta) for the function: $f(n) = 2n^3 + 4n + 5$ Justify each bound with proper mathematical proof.

[4 Marks]

(b) Explain the concept of recurrence relations and their role in analyzing recursive algorithms. Solve the recurrence $T(n) = T(n/2) + 1$ using the Master Theorem and relate it to a standard algorithm.

[3 Marks]

Q2 [Divide and Conquer | Greedy Algorithms] [8 Marks]

(a) Apply the Quick Sort algorithm on the following array. Choose the last element as pivot and show all partitioning steps. Compute the time complexity for this case.
Array: [7, 2, 9, 4, 3, 1, 5]

[4 Marks]

(b) Use the Greedy algorithm to solve the Job Scheduling problem for the following instance. Show the schedule and compute maximum profit. $N = 4$

Jobs: J1 J2 J3 J4

Profits: 30 35 20 25

Deadline: 2 1 2 1

[4 Marks]

Q3 [Dynamic Programming] [7 Marks]

(a) Use Dynamic Programming to solve the Sum of Subset problem for the following set and target sum. Show the DP table completely. Set: {3, 34, 4, 12, 5, 2} | Target Sum: 9

[4 Marks]

(b) Explain the principle of optimality in Dynamic Programming. How does it distinguish DP from the greedy approach? Illustrate with a suitable example.

[3 Marks]

Q4 [Branch and Bound | Graph Algorithms] [8 Marks]

(a) Apply the FIFO Branch and Bound method to solve the following 0/1 Knapsack problem. Draw the state-space tree and show how upper bounds are computed at each node. Knapsack Capacity: $W = 10$ Items — Weights: (4, 5, 1) Values: (40, 30, 20)

[4 Marks]

(b) Apply BFS (Breadth First Search) on the following graph starting from vertex 1. Show the BFS tree and state the order of vertices visited. Also write the time complexity of BFS.

Graph (Adjacency): 1–2, 1–3, 2–4, 2–5, 3–6, 3–7

[4 Marks]

Sample Paper 3

Q1 [Divide and Conquer | Time Complexity] [7 Marks]

(a) Apply the Karatsuba Divide and Conquer algorithm to multiply the following two large integers. Show all three sub-multiplications and the combine step. $X = 1234$, $Y = 5678$ Also derive the recurrence relation and solve it using the Master Theorem to find the time complexity.

[4 Marks]

(b) Compare Quick Sort and Merge Sort with respect to best-case, worst-case, and average-case time complexities and space complexity. Which is preferred for large datasets with random input and why?

[3 Marks]

Q2 [Greedy Algorithms | Graph Algorithms – MST] [8 Marks]

(a) Apply Kruskal's Algorithm to find the Minimum Spanning Tree (MST) for the following weighted graph. Show the edge selection steps using union-find and state the total MST cost. Edges: A–B(4), A–C(3), B–C(1), B–D(2), C–D(4), C–E(6), D–E(5)

[4 Marks]

(b) Analyze the time complexity of Dijkstra's Algorithm. Why is it more efficient than Bellman-Ford for graphs with non-negative edge weights? Compare both algorithms.

[4 Marks]

Q3 [Dynamic Programming] [7 Marks]

(a) Use Dynamic Programming to compute the Binomial Coefficient $C(6, 3)$. Draw the complete Pascal's triangle DP table. Compare the time complexity of DP vs. the recursive approach.

[4 Marks]

(b) Explain the role of memoization in Dynamic Programming. How does it improve performance compared to plain recursion? Illustrate using the Fibonacci sequence example.

[3 Marks]

Q4 [Backtracking | Branch and Bound] [8 Marks]

(a) Solve the following 0/1 Knapsack problem using the Backtracking approach. Draw the complete state-space tree. Clearly indicate pruned nodes and the optimal solution. Knapsack Capacity: $M = 5$ Items — Weights: (2, 1, 3, 2) Profits: (12, 10, 20, 15)

[4 Marks]

(b) Explain the concept of Branch and Bound. Differentiate between FIFO, LIFO, and Least-Cost (LC) strategies for node exploration. When would you prefer LC Branch and Bound over FIFO?

[4 Marks]

Sample Paper 4

Q1 [Asymptotic Analysis | Time Complexity] [7 Marks]

- (a) Solve the following recurrence relations using the Master Theorem. For each, identify the applicable case and state the final time complexity. (i) $T(n) = 8T(n/2) + n^2$
(ii) $T(n) = 4T(n/2) + n^2$ (iii) $T(n) = 2T(n/2) + n \log n$ (check applicability) [4 Marks]

- (b) Explain the relationship between P, NP, NP-Hard, and NP-Complete problem classes. Draw a Venn diagram. Give one example problem for each class. [3 Marks]

Q2 [Divide and Conquer | Greedy Algorithms] [8 Marks]

- (a) Apply Binary Search to find the position of element 9 in the following sorted array. Show every comparison step. Write the Binary Search algorithm and analyze its time complexity using a recurrence relation. Array: [2, 3, 5, 7, 9, 11, 13, 15] [4 Marks]

- (b) Use the divide and conquer approach to find the Maximum Subarray for the array below. Show the divide, conquer, and combine steps clearly. Array: [-2, 1, -3, 4, -1, 2, 1, -5, 4] [4 Marks]

Q3 [Dynamic Programming | Graph Algorithms] [7 Marks]

- (a) Apply the Bellman-Ford Algorithm on the following graph starting from vertex A. Show the distance table after each iteration. Also detect if any negative weight cycle exists. Edges: $A \rightarrow B(3)$, $B \rightarrow C(2)$, $C \rightarrow A(-6)$, $A \rightarrow D(5)$ [4 Marks]

- (b) Analyze the difference between the Floyd-Warshall algorithm and Dijkstra's algorithm for finding shortest paths. State the time complexity of each and mention one scenario where Floyd-Warshall is preferred. [3 Marks]

Q4 [Backtracking | Dynamic Programming] [8 Marks]

- (a) Using Backtracking, place 4 queens on a 4×4 chessboard such that no two queens attack each other. Write the pseudocode. Show the step-by-step recursive calls and all valid placement(s). [4 Marks]

(b) Solve the Multistage Graph problem using Dynamic Programming for the following graph. Find the minimum cost path from source S to destination T. Edges:
S→A(2), S→B(4), A→C(3), A→D(6), B→C(5), B→D(2), C→T(4), D→T(1)

[4 Marks]

Sample Paper 5

Q1 [Time Complexity | Greedy Strategy] [7 Marks]

(a) Compare and evaluate the time complexities of Merge Sort and Quick Sort across best, worst, and average cases. For each, write the recurrence relation and solve it. Conclude which algorithm is more suitable for real-world sorting scenarios.

[4 Marks]

(b) Explain the control abstraction (generic structure) of the Greedy Strategy. What are the key properties a problem must satisfy for a greedy algorithm to yield an optimal solution?

[3 Marks]

Q2 [Graph Algorithms – BFS/DFS | MST] [8 Marks]

(a) Apply Prim's Algorithm to find the Minimum Spanning Tree (MST) for the following weighted undirected graph. Start from vertex A. Show the MST growth at each step and the final MST cost. Edges: A–B(2), A–D(6), B–C(3), B–D(8), B–E(5), C–E(7), D–E(9)

[4 Marks]

(b) Apply DFS (Depth First Search) on the following graph starting from vertex 1. Show the DFS tree, discovery and finish times for each vertex. State the time complexity. Graph (Adjacency): 1–2, 1–3, 2–4, 3–4, 4–5

[4 Marks]

Q3 [Dynamic Programming] [7 Marks]

(a) Find the Longest Common Subsequence (LCS) of the following two sequences using Dynamic Programming. Fill the complete DP table and trace back to identify the LCS string. X = "ABCB DAB" Y = "BDCABA"

[4 Marks]

(b) Explain why the 0/1 Knapsack problem cannot be solved optimally using the Greedy approach. Support your answer with a counter-example. How does Dynamic Programming overcome this limitation?

[3 Marks]

Q4 [Backtracking | Branch and Bound] [8 Marks]

(a) Use Backtracking to find all subsets of the following set that sum to the given target. Draw the state-space tree and prune branches where the remaining sum exceeds the target. Set: {1, 3, 9, 2, 7} | Target Sum: 10

[4 Marks]

(b) Explain the Least Cost (LC) Branch and Bound strategy. Apply it to find the optimal solution for the following Traveling Salesman Problem (TSP) with 3 cities using the given cost matrix:

To: C1 C2 C3

C1: [∞ 10 5]

C2: [8 ∞ 9]

C3: [4 6 ∞]

[4 Marks]
